

Modulparameter, Kernmeldungen und Systemintegration

Kernelmodule konfigurieren, beobachten und sauber einbinden

2.04

Konfiguration und Beobachtung verbinden

01 · PARAMETER

Verhalten steuern

- Werte beim Laden setzen
- Schreibbare Parameter zur Laufzeit ändern

02 · MELDUNGEN

Zustand erkennen

- Übergänge sichtbar machen
- Fehler und relevante Werte protokollieren

03 · INTEGRATION

Betrieb wiederholen

- Abhängigkeiten auflösen
- Defaults und Autoload getrennt verwalten

Steuerung wird erst durch Beobachtung und reproduzierbare Integration beherrschbar.

Kontrollierte Betriebsarten ohne Neubau

BUILD

Default

- Im Quellcode definiert
- Sicherer Ausgangswert

START

Beim Laden

- Wert an insmod oder modprobe übergeben

BETRIEB

Zur Laufzeit

- Über sysfs ändern, wenn der Parameter schreibbar ist

Geeignet für Intervalle, Schalter und Limits. Komplexe oder geheime Konfiguration gehört nicht in Modulparameter.

module_param(): Name, Typ und Rechte

QUELLCODE

```
static int interval_ms = 1000;  
module_param(interval_ms, int, 0644);  
MODULE_PARM_DESC(interval_ms, "Intervall in Millisekunden");
```

01 Name

Variable und sichtbarer
Parametername

02 Typ

Parser für int, bool, charp und
weitere Typen

03 Rechte

0000 unsichtbar,
0444 lesen, 0644 schreiben

Die Rechte steuern die sysfs-Sichtbarkeit, nicht die Übergabe beim Laden.

Parametertypen und Darstellung

`bool`

true/false oder 1/0

Schalter

`int / uint`

Ganzzahl

Intervall oder Limit

`charp`

Zeichenkette

Name oder Modus

`module_param_array`

Kommagetrennte Werte

Kleine Werteliste

BEISPIEL

```
module_param_array(levels, int, &count, 0444);
```

Der Parametertyp legt Parsing und Darstellung fest.

Parameter beim Modulstart übergeben

TERMINAL

```
$ sudo insmod ./demo.ko interval_ms=500 verbose=1  
$ sudo modprobe demo interval_ms=500 verbose=1
```

01`insmod ./demo.ko`

Direkter Pfad ohne Abhängigkeitsauflösung

02`modprobe demo`

Modulbaum, Abhängigkeiten und Optionen

03`modinfo -p demo`

Deklarierte Parameter und Beschreibungen prüfen

modprobe nutzt den Modulbaum, insmod lädt genau die angegebene Datei.

Modulparameter im sysfs

TERMINAL

```
$ ls /sys/module/demo/parameters
$ cat /sys/module/demo/parameters/interval_ms
$ echo 750 | sudo tee /sys/module/demo/parameters/interval_ms
```

01 READ-ONLY · 0444

Im sysfs sichtbar und nach dem Laden nur lesbar.

02 READ-WRITE · 0644

Der Eigentümer darf den Wert zur Laufzeit ändern.

Ladeoption und sysfs-Berechtigung bleiben voneinander getrennt.

Validierung mit Parameter-Callbacks

QUELLCODE

```
static int set_interval( const char *val,  
                        const struct kernel_param *kp)  
{  
    int n, ret = kstrtoint(val, 0, &n);  
    if (ret || n < 10 || n > 60000)  
        return -EINVAL;  
    return param_set_int(val, kp);  
}
```

01 Parsen

Text sicher in den Zieltyp umwandeln.

02 Prüfen

Den fachlich gültigen Bereich explizit begrenzen.

03 Übernehmen

Nur einen gültigen Wert veröffentlichen.

Der Callback verhindert ungültige Zustände vor der Übernahme.

Schreibbare Parameter und Nebenläufigkeit

- | | | | |
|----|----------------------|--|------------------------------------|
| 01 | Ein Skalar | <code>READ_ONCE(value)</code> | Konsistenter Einzelzugriff |
| 02 | Mehrere Felder | <code>mutex</code> / <code>spinlock</code> | Zusammengehörigen Zustand schützen |
| 03 | Änderung mit Wirkung | <code>Callback</code> + <code>Lock</code> | Prüfen und atomar anwenden |

volatile ersetzt weder Atomarität noch Synchronisation.

Log-Level für Kernelmeldungen

<code>pr_err()</code>	Fehler	Aktion fehlgeschlagen
<code>pr_warn()</code>	Warnung	Unerwarteter Zustand, Ausführung läuft weiter
<code>pr_info()</code>	Information	Wichtiger Übergang
<code>pr_debug()</code>	Diagnose	Nur bei Bedarf sichtbar

Im Kernel ersetzt das Log die Standardausgabe.

Anatomie einer guten Kernelmeldung

QUELLE

Wo?

Eindeutiges Präfix oder Gerätename

EREIGNIS

Was?

Aktion und Ergebnis in klarer Sprache

KONTEXT

Warum?

Fehlercode und relevanter Wert

BEISPIEL

```
pr_err("demo: start failed: interval=%d err=%d\n",  
       interval_ms, ret);  
pr_info_ratelimited("demo: event dropped\n");
```

Präfix, Ereignis und Kontext machen eine Meldung auffindbar und verwertbar.

Kernelmeldungen verfolgen und filtern

- | | | |
|----|--|--|
| 01 | <code>sudo dmesg -w</code> | Neue Kernelmeldungen fortlaufend anzeigen |
| 02 | <code>sudo journalctl -k -f</code> | Kernelmeldungen im systemd-Journal verfolgen |
| 03 | <code>dmesg --level=err,warn</code> | Nur Fehler und Warnungen anzeigen |
| 04 | <code>sudo dmesg grep 'demo:'</code> | Nach dem Modulpräfix filtern |

`journalctl -k` setzt `systemd` voraus. Der Logzugriff folgt der Systemrichtlinie.

Dynamic Debug ohne Neubau

TERMINAL

```
# verfügbare Aufrufstellen ansehen
$ sudo grep demo /sys/kernel/debug/dynamic_debug/control

# Meldungen für das Modul ein- und ausschalten
$ echo 'module demo +p' | sudo tee /sys/kernel/debug/dynamic_debug/control
$ echo 'module demo -p' | sudo tee /sys/kernel/debug/dynamic_debug/control
```

Voraussetzung sind CONFIG_DYNAMIC_DEBUG und ein eingehängtes debugfs.

Modul mit depmod und modprobe integrieren

TERMINAL

```
$ sudo install -D -m 0644 demo.ko /lib/modules/$(uname -r)/extra/demo.ko
$ sudo depmod -a
$ sudo modprobe demo
$ sudo modprobe -r demo
```

01 Installieren

demo.ko installieren

02 DB aktualisieren

depmod -a

03 Laden

modprobe <module>

04 Entladen

modprobe -r <module>

Distribution, Paketierung und Secure-Boot-Richtlinien des Zielsystems bleiben maßgeblich.

Optionen und Autoload getrennt verwalten

```
/etc/modprobe.d/demo.conf
```

```
options demo interval_ms=500 verbose=1
```

```
/etc/modules-load.d/demo.conf
```

```
demo
```

01

Parameter

Werte validiert und sichtbar

02

Meldungen

Nützlich, filterbar und begrenzt

03

Integration

modprobe arbeitet reproduzierbar

Parameter steuern das Modul. Autoload entscheidet, wann es geladen wird.